

Experiment 'p_6_1_b_process_array_every_even_element_seq'

Results

June 4, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing assume : 0
Responses containing expect : 0
Resolution attempts: 2
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_6_1_b_process_array_every_even_element_seq

Natural language statement: Write a method that takes an array of ten integers and then returns every even element.

Method signature: p_6_1_b_process_array_every_even_element_seq(inputs: seq<int>) returns (even_elements: seq<int>)

Ensures

- even_elements == evens(inputs)

Requires

- |inputs| == 10

Functional Code Given

```
function evens(inputs: seq<int>): seq<int>
{
  if inputs == [] then []
  else if inputs[0] % 2 == 0 then [inputs[0]] + evens(inputs[1..])
  else evens(inputs[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes an array of ten integers and then returns every even element.

The signature should be:

p_6_1_b_process_array_every_even_element_seq(inputs: seq<int>) returns (even_elements:
→ seq<int>)

The method should respect the following contract:

requires |inputs| == 10, ensures even_elements == evens(inputs)

The contract uses the following dafny code:

```
function evens(inputs: seq<int>): seq<int>
{
  if inputs == [] then []
  else if inputs[0] % 2 == 0 then [inputs[0]] + evens(inputs[1..])
  else evens(inputs[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn006Sv2Tvdq9AO9v2YSr1rSjK0XV

Model: gpt-5.5-2026-04-23

Created at: 1780570446

Finish reason: stop

Usage: 454 tokens in, and 850 tokens out

```
{
  even_elements := [];

  if inputs[0] % 2 == 0 {
    even_elements := even_elements + [inputs[0]];
  }
  if inputs[1] % 2 == 0 {
    even_elements := even_elements + [inputs[1]];
  }
}
```

```

if inputs[2] % 2 == 0 {
    even_elements := even_elements + [inputs[2]];
}
if inputs[3] % 2 == 0 {
    even_elements := even_elements + [inputs[3]];
}
if inputs[4] % 2 == 0 {
    even_elements := even_elements + [inputs[4]];
}
if inputs[5] % 2 == 0 {
    even_elements := even_elements + [inputs[5]];
}
if inputs[6] % 2 == 0 {
    even_elements := even_elements + [inputs[6]];
}
if inputs[7] % 2 == 0 {
    even_elements := even_elements + [inputs[7]];
}
if inputs[8] % 2 == 0 {
    even_elements := even_elements + [inputs[8]];
}
if inputs[9] % 2 == 0 {
    even_elements := even_elements + [inputs[9]];
}
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_6_1_b_process_array_every_even_element_seq/program.dfy(11,0): Error: a
  ↳ postcondition could not be proved on this return path

```

```

|
11 | {
|   ^

```

```

problem_data/p_6_1_b_process_array_every_even_element_seq/program.dfy(10,23): Related
  ↳ location: this is the postcondition that could not be proved

```

```

|
10 | ensures even_elements == evens(inputs)
|   ^^

```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↳ Dafny body, including the curly braces that surround it. Do not show the signature
 ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn00vVvZb65EIKhh4A00xf8Umqfc4

Model: gpt-5.5-2026-04-23

Created at: 1780570497

Finish reason: stop

Usage: 954 tokens in, and 3418 tokens out

```
{
  even_elements := [];
  var i := 0;

  assert inputs[0..] == inputs;
  assert even_elements + evens(inputs[i..]) == evens(inputs);

  while i < |inputs|
    invariant 0 <= i <= |inputs|
    invariant even_elements + evens(inputs[i..]) == evens(inputs)
    decreases |inputs| - i
  {
    assert inputs[i..] != [];
    assert (inputs[i..])[0] == inputs[i];
    assert (inputs[i..])[1..] == inputs[i + 1..];

    if inputs[i] % 2 == 0 {
      assert evens(inputs[i..]) == [inputs[i]] + evens(inputs[i + 1..]);
      var prev := even_elements;
      assert prev + evens(inputs[i..]) == evens(inputs);
      even_elements := even_elements + [inputs[i]];
      assert even_elements == prev + [inputs[i]];
      assert even_elements + evens(inputs[i + 1..]) == prev + ([inputs[i]] + evens(inputs[
        ↪ i + 1..]));
      assert prev + ([inputs[i]] + evens(inputs[i + 1..])) == prev + evens(inputs[i..]);
      assert even_elements + evens(inputs[i + 1..]) == evens(inputs);
    } else {
      assert evens(inputs[i..]) == evens(inputs[i + 1..]);
      assert even_elements + evens(inputs[i + 1..]) == evens(inputs);
    }

    i := i + 1;
  }

  assert i == |inputs|;
  assert inputs[i..] == [];
  assert evens(inputs[i..]) == [];
  assert even_elements == even_elements + [];
  assert even_elements == evens(inputs);
}
```

Final Program

```

function evens(inputs: seq<int>): seq<int>
{
  if inputs == [] then []
  else if inputs[0] % 2 == 0 then [inputs[0]] + evens(inputs[1..])
  else evens(inputs[1..])
}

method p_6_1_b_process_array_every_even_element_seq(inputs: seq<int>) returns (
  ↪ even_elements: seq<int>)
  requires |inputs| == 10
  ensures even_elements == evens(inputs)
{
  even_elements := [];
  var i := 0;

  assert inputs[0..] == inputs;
  assert even_elements + evens(inputs[i..]) == evens(inputs);

  while i < |inputs|
    invariant 0 <= i <= |inputs|
    invariant even_elements + evens(inputs[i..]) == evens(inputs)
    decreases |inputs| - i
    {
      assert inputs[i..] != [];
      assert (inputs[i..])[0] == inputs[i];
      assert (inputs[i..])[1..] == inputs[i + 1..];

      if inputs[i] % 2 == 0 {
        assert evens(inputs[i..]) == [inputs[i]] + evens(inputs[i + 1..]);
        var prev := even_elements;
        assert prev + evens(inputs[i..]) == evens(inputs);
        even_elements := even_elements + [inputs[i]];
        assert even_elements == prev + [inputs[i]];
        assert even_elements + evens(inputs[i + 1..]) == prev + ([inputs[i]] + evens(inputs[
          ↪ i + 1..]));
        assert prev + ([inputs[i]] + evens(inputs[i + 1..])) == prev + evens(inputs[i..]);
        assert even_elements + evens(inputs[i + 1..]) == evens(inputs);
      } else {
        assert evens(inputs[i..]) == evens(inputs[i + 1..]);
        assert even_elements + evens(inputs[i + 1..]) == evens(inputs);
      }

      i := i + 1;
    }

  assert i == |inputs|;
  assert inputs[i..] == [];
  assert evens(inputs[i..]) == [];
  assert even_elements == even_elements + [];
  assert even_elements == evens(inputs);
}

```

Total Token Usage

Input tokens: 1408

Output tokens: 4268

Reasoning tokens: 3457

Sum of ‘total tokens’: 5676

Experiment Timings

Overall Experiment started at 1780570444445, ended at 1780570592287, lasting 147842ms (147.84 seconds)

Iteration #1 started at 1780570444446, ended at 1780570496960, lasting 52514ms (52.51 seconds)

Iteration #2 started at 1780570497023, ended at 1780570592287, lasting 95264ms (95.26 seconds)